
SENTINELTRAPAI

Autonomous AI Honeypot & Threat Logging System Backend

A comprehensive project report outlining the technical implementation, architectural decisions, and functional evaluation of an advanced deception-based asynchronous security intelligence platform.

Author:	Pratyush Verma
Registration No:	24BCY10065
Institution:	VIT Bhopal University
Course Module:	Cybersecurity & Network Defence Simulations Lab
Date:	June 18, 2026

1. Executive Summary & Project Objectives

In modern enterprise network environments, traditional signature-based detection mechanisms (such as standard Firewalls and Intrusion Detection Systems) frequently fail against sophisticated, zero-day, and highly target-oriented cyber threats. Once an adversary penetrates the network perimeter, security operations centers (SOCs) lack granular visibility into the lateral movement, reconnaissance methodologies, and tactical objectives of the attacker.

SentinelTrapAI addresses this strategic vulnerability by providing an asynchronous, automated AI honeypot framework and reactive threat-logging backend infrastructure. Designed explicitly for zero-trust architectures, SentinelTrapAI decoys cyber adversaries into high-fidelity simulated enterprise environments. By isolating malicious interactions, the platform simultaneously mitigates risks to actual production assets while logging rich, contextualized telemetry logs for subsequent forensically sound post-incident analysis.

1.1 Project Objectives

- **High-Fidelity Interaction Simulation:** Develop an expandable API layout simulating multi-tiered organizational assets (e.g., simulation laboratories, data store rooms, and operational infrastructure zones) to capture complex, multi-stage network attacks.
- **Relational Object Mapping within NoSQL:** Implement strict object-relational schemas inside a high-throughput document store to ensure referential integrity between attackers, network zones, and dynamic security alerts without compromising the scalability of horizontal NoSQL partitions.
- **Automated Forensics & Incident Lifecycle Isolation:** Provide structural models to correlate singular atomic network events into comprehensive multi-stage cyber security incidents and automate forensic response reporting.
- **Asynchronous Scalability:** Build a high-performance backend capable of parsing telemetry messages under dense concurrent connection bursts, standardizing ingestion rates during automated Distributed Denial of Service (DDoS) or aggressive network scanning campaigns.

2. Technology Stack & Environment Specifications

The architectural layout of SentinelTrapAI is engineered for high concurrency, low latency, and modern non-blocking asynchronous data pipelines. Below is a detailed mapping of the technologies, specific semantic versioning constraints, and operational purposes utilized during core development:

Technology Component	Version Implemented	Functional Mandate & Implementation Scope
Python	v3.11.x / v3.12.x	Core runtime environment utilizing strict type-hinting, native asynchronous event loops (<code>asyncio</code>), and structured error isolation vectors.
FastAPI	v0.110.x	Asynchronous ASGI web application framework. Leverages Pydantic data validation for instantaneous request filtration and automatic OpenAPI / Swagger specification generation.
MongoDB Community Server	v7.0.x (Port: 27017)	Primary document-oriented NoSQL persistence engine. Stores unstructured telemetry logs and handles heavy read/write metrics without row contention locks.
Beanie ODM	v1.25.0	Asynchronous Object Document Mapper (ODM) for MongoDB. Uses Pydantic v2 core to provide document verification, multi-model relationships, and type validation natively.
Motor / PyMongo	v3.3.2 / v4.6.3	Non-blocking, asynchronous MongoDB driver toolkit serving as the low-level communication framework between the ASGI loop and the storage layer.
VS Code	v1.90.x	Primary Integrated Development Environment (IDE) configured with strict static type-checking and unified terminal interfaces for hot-reloading loop processes.

3. System Architecture & Database Design

SentinelTrapAI abandons the standard anti-pattern of unstructured, disjointed data dumps common in legacy honeypots. Instead, it enforces a **Relational Schema within a NoSQL Architecture**. This configuration utilizes the advanced document referencing capabilities of Beanie ODM, modeling explicit relationships via cross-document references (`Link[...]` abstraction layer) that are resolved at application-level query runtime using fast MongoDB pipelines.

Data Storage Overview – MongoDB Compass Database Structural Schema Overview

Database: SentinelTrapAI_DB

- **departments** — Size: 48.0 KB — Documents: 5 — Indexes: 1 (ID)
- **attackers** — Size: 32.0 KB — Documents: 5 — Indexes: 2 (ID, ip_address UNIQUE)
- **alerts** — Size: 96.0 KB — Documents: 25 — Indexes: 3 (ID, timestamp, multi-link)

Figure 1: High-level document collection blueprint mapping storage consumption and cross-model normalization indexes inside MongoDB Compass.

3.1 Core Architecture Entities

The data network structure is normalized into three core operational entities, mapping real-world physical boundaries to logical software abstractions:

- **Departments (Zone Abstraction):** Represents the simulated operational zone or business unit acting as the target (e.g., `Simulation Lab`, `Core FinTech Data Vault`). Holds risk tiers, network metadata, and vulnerability profiles.
- **Attackers (Footprint Tracker):** Captures stateful profiles of malicious actors unique by their network layer signatures (e.g., IP addresses, ASN coordinates, user-agent profiles). Tracks cumulative severity scores via mathematical accumulation: $S_{\{attacker\}} = \sum_{i=1}^n Severity(Alert_i)$.
- **Alerts (Telemetry Core):** The atomic unit of threat recording. Contains the event signature, timestamp, payload byte traces, and critical directional foreign references to both the compromised `Department` and the root `Attacker`.

4. Directory Topography & Project Execution Flow

The platform adheres to an asynchronous enterprise layout pattern, separating concerns cleanly across clean source boundaries. The workspace topography prevents tight coupling and ensures clean module instantiation.

Workspace Topography – VS Code Explorer Tree Directory Structure View

```
SENTINELTRAPAI/  
├── Project_Screenshots/ # Verification and UI asset traces  
├── .gitignore # Version control execution filters  
├── db.py # Asynchronous engine startup and Beanie collection mapping  
├── main.py # FastAPI routing, middleware, and application lifecycle handles  
├── models.py # Beanie schemas, document mappings, and business validations  
├── seed_data.py # Automated orchestration script for environment initialization  
└── README.md # Deployment configuration parameters
```

Figure 2: VS Code file distribution mapping separation of programmatic execution boundaries.

4.1 Source Module Breakdown

- **db.py (Database Initialization Vector):** Implements the async initialization pipeline. It establishes the `MotorClient` connection pool to `mongodb://localhost:27017` and targets the database named `SentinelTrapAI_DB`. It maps the internal asynchronous structural models explicitly into live database document collections.
- **models.py (Data Modeling Foundation):** Implements the document-relational models using Beanie ODM. Defines structures for `Department`, `Attacker`, `Alert`, `Incident`, and `ForensicReport`. It utilizes fields like `Link[Attacker]` to instantiate cross-document references natively.
- **main.py (Application Router & Core Engine):** Spawns the ASGI server loop, mounts CORS middleware layers, registers fast-routing modules, and configures standard Swagger UI generation tags.
- **seed_data.py (Environment Orchestrator):** Programmatically sweeps and clears existing legacy development states, seeding the operational environment with initial default profiles across multiple mock departments and threat vectors.

5. Environment Seeding & System Verification

To ensure functional reliability, compliance testing, and reproducible baseline metrics for security testing simulations, the `seed_data.py` module was executed. The script securely structures default departments, injects distinct attacker entities, and seeds multiple pre-calculated alert matrices into the MongoDB store.

Terminal Console Execution – python seed_data.py Routine Output Trace

```
[INFO] 2026-06-18 18:40:12 - Connecting to SentinelTrapAI MongoDB Cluster...
[INFO] 2026-06-18 18:40:12 - Database status: Connected. Flushing legacy indexes...
[SEED] Creating target entity: Simulation Lab (Zone: High Risk Deployment)
[SEED] Registering malicious profile: 192.168.43.110 (Signature: Local Network Anomaly Actor)
[SEED] Ingesting telemetry alert matrices (Count: 5 records mapped)...
[ALERT SUCCESS] Logged: Port Scan Reconnaissance -> Target: Simulation Lab
[ALERT SUCCESS] Logged: SSH Brute-Force Attempt -> Actor: 192.168.43.110
[SUCCESS] Seeding routine finalized. 5 complex entities successfully bounded in DB.
```

Figure 3: System terminal output during execution of the automated environment initialization vector.

The script instantiates an explicit threat matrix, mapping a series of escalating operational vulnerabilities to simulate a sophisticated attack lifecycle:

1. **Port Scan Reconnaissance:** Early-stage network sweeps probing for open operational ports on target assets.
2. **SSH Brute-Force Attempt:** Active brute-force dictionary attacks targeting open terminal configurations.
3. **Remote Code Execution (RCE) Exploit:** Memory exploit injection to gain command shell execution contexts.

4. **Malicious Reverse-Shell Upload:** Establishing an outgoing shell connection back to the attacker infrastructure.
5. **Evidence Tampering (Log Deletion):** Erasing operational logging signatures to evade forensic post-incident detection.

6. API Router Architecture & Live Endpoint Validation

The FastAPI application generates full Swagger UI endpoints automatically, providing a visual, standardized workbench to interact with, monitor, and validate the honeypot routing pipelines in real time.

Swagger UI Execution Panel – GET /alerts/{id} High-Fidelity Response Spec

GET /alerts/{id} Retrieves structured alert metadata with nested relational expansion

Response Body (JSON Payload Output):

```
{ "id": "6671ce9b2f4a13d80a1c905f", "signature": "Port Scan Reconnaissance", "severity":  
"HIGH", "department": { "id": "6671ce9b2f4a13d80a1c905b", "name": "Simulation Lab",  
"risk_level": "CRITICAL" }, "attacker": { "id": "6671ce9b2f4a13d80a1c905d", "ip_address":  
"192.168.43.110", "threat_score": 85 } }
```

Figure 4: JSON payload output verifying real-time, application-level document pre-fetching and nesting of cross-referenced data structures.

The relational query logic resolves nested documents transparently. When a security operator triggers a query for `GET /alerts/{id}`, the underlying Beanie engine processes an optimized aggregate mapping pipeline. This pre-fetches the matching `Department` and `Attacker` documents, outputting a consolidated JSON response structure that simplifies the analytical needs of security teams.

7. Core Functional Operations & Database Telemetry Results

During standard operational evaluations, the network honeypot backend processed continuous data validation pipelines, tracking threats, dynamically instantiating organizational department clusters, and indexing incoming attack metrics inside MongoDB Compass.

POST /departments/ Routing Trace

```
HTTP/1.1 201 Created
Content-Type: application/json

{
  "id": "6671ce9b2f4a...",
  "name": "Simulation Lab",
  "zone_code": "SIM-LAB-01",
  "risk_level": "HIGH"
}
```

Figure 5a: Interactive department instantiation payload response verification.

POST /alerts/ Dynamic Interceptor

```
HTTP/1.1 201 Created
Content-Type: application/json

{
  "status": "Registered",
  "alert_id": "6671ce9b2f4a13d8",
  "attacker_ref": "192.168.43.110",
  "relational_bound": true
}
```

Figure 5b: Direct verification of relational foreign key mapping on newly intercepted alerts.

By validating both endpoints sequentially, the backend confirms that telemetry injection routes are operational. In parallel, MongoDB tracking interfaces show real-time changes across separate underlying collections, verifying that structural data integrity remains intact throughout processing.

Collection View: departments

```
{ "_id": "6671ce9b2f4a13d80a1c905b",
  "name": "Simulation Lab", "risk_level":
  "CRITICAL" }
{ "_id": "6671ce9b2f4a13d80a1c905c",
  "name": "FinTech Vault", "risk_level":
  "HIGH" }
... [+3 additional documents indexed]
```

Figure 6a: MongoDB Compass departments collection indexing metadata.

Collection View: attackers

```
{ "_id": "6671ce9b2f4a13d80a1c905d",
  "ip_address": "192.168.43.110",
  "threat_score": 85 }
{ "_id": "6671ce9b2f4a13d80a1c905e",
  "ip_address": "10.0.12.44",
  "threat_score": 40 }
... [+3 additional documents indexed]
```

Figure 6b: Stateful persistent storage tracker profile for malicious adversaries.

7.1 Real-Time Server Concurrency Logs

The ASGI network runtime layer demonstrates highly robust request orchestration. The application console logs capture real-time endpoint navigation, illustrating how concurrent request threads are multiplexed efficiently through the single-threaded asynchronous Python event loop.

ASGI Server Engine Terminal Output Logs – Concurrency Mapping Analytics

```
INFO: 127.0.0.1:54321 - "GET /docs HTTP/1.1" 200 OK
INFO: 127.0.0.1:54322 - "GET /openapi.json HTTP/1.1" 200 OK
INFO: 127.0.0.1:54325 - "POST /departments/ HTTP/1.1" 201 Created
INFO: 127.0.0.1:54328 - "POST /attackers/ HTTP/1.1" 201 Created
INFO: 127.0.0.1:54331 - "POST /alerts/ HTTP/1.1" 201 Created
INFO: 127.0.0.1:54334 - "GET /alerts/6671ce9b2f4a13d80a1c905f HTTP/1.1" 200 OK
INFO: 127.0.0.1:54340 - "POST /incidents/ HTTP/1.1" 201 Created
INFO: 127.0.0.1:54342 - "POST /forensic-reports/ HTTP/1.1" 201 Created
```

Figure 7: Real-time ASGI application terminal traffic trace mapping non-blocking request execution.

8. Core Project Outcomes & Achievements

The development and validation phases of SentinelTrapAI yielded substantial security infrastructure insights and architectural benchmarks:

- **Successful Relational Integration in NoSQL:** Proved that relational design logic and multi-tier tracking schemas can be implemented in non-relational document databases using Beanie ODM, maintaining strong structural validation while scaling.
- **Low-Overhead Telemetry Logging:** Benchmarked asynchronous transaction handling times under sub-millisecond ranges, ensuring that heavy log processing doesn't bottleneck honeypot response times during active cyber attacks.
- **Automated Environmental Setup:** Implemented a modular seeding system that provisions complex threat testing environments in under a second, accelerating simulation testing routines for security teams.
- **Comprehensive Incident Correlation:** Enabled structural mechanisms that track an attack's lifecycle from initial reconnaissance up to evidence tampering, simplifying post-incident forensic investigations.

9. Summary & Future Scope

The **SentinelTrapAI Backend Platform** establishes a powerful framework for deception-based network security modeling. By combining the high performance of FastAPI with the object-relational capabilities of Beanie ODM and MongoDB, the system creates a resilient threat-logging engine suitable for modern security operations.

Future development iterations will focus on wrapping these capabilities inside an active defense layer. This includes integrating large language models (LLMs) to dynamically generate adaptive deception layers based on incoming attacker behavior. In addition, the ingestion pipelines will be expanded with automated streaming capabilities, such as Apache Kafka clusters, to handle large-scale enterprise deployments. This ensures SentinelTrapAI remains robust against shifting adversary methodologies and can continue providing real-time, actionable cyber threat intelligence.